



Project Launch Runbook

Cowork multi-session project bootstrap

From “create a project” to the coordinator’s “team up & ready” report

For the Operator, Coordinator, and Curator

Edition EN · 2026-06-11 · Reusable, product-agnostic methodology

Document revision history

This table lists the revision history of this document.

Version	Date	Summary of changes	Product release ID	Shipped-with (commit/barrier)
1.0	2026-06-11	initial runbook	RTM-REL-2026.06	(pending push)

Contents

Document revision history.....	2
Contents.....	3
Purpose & audience.....	4
How to use this runbook.....	4
1. Roles & responsibilities.....	4
2. The six-phase model.....	4
3. Phases.....	5
P0 — Universe (the empty world).....	5
P1 — Core (discipline + bus).....	5
P2 — Roster & Claims.....	6
P3 — Tree hygiene.....	6
P4 — Work-done map.....	6
P5 — Ready.....	7
4. Operational loop — task dispatch (the heartbeat after launch).....	7
5. Cross-cutting invariants (binding throughout).....	8
6. Case study & lessons — Agent Desktop bootstrap.....	8
Timeline.....	9
Lessons (generalize for any project).....	9
Tree-hygiene numbers (from the worked example).....	9
Appendix A — CLAUDE.md skeleton.....	10
Appendix B — .coord/ bus files.....	10
Appendix C — Init-prompt template (per specialist).....	11
Appendix D — Claim-map template.....	11
Appendix E — “Team up & ready” report template.....	12

Purpose & audience

This runbook captures the proven process for launching a new Cowork project as a multi-session team: one operator, one coordinator, an optional external curator, and a set of specialist sessions. It exists so a new project can reach a credible “team up & ready” state quickly and safely, with the same discipline that keeps a long-running project honest.

Audience: the Operator (directs the project), the Coordinator (owns the bus and the gates), the Curator (external source-of-truth / certifier, when present), and specialists being onboarded.

How to use this runbook

- **Work the six phases P0 → P5 in order.** Each phase has a Goal · Who · Actions · Artifacts · Verification gate · Operator GO.
- **Every gate is verified against the object store / repo walk**, never from a chat claim.
- Copy the Appendix templates (A–E) and fill the placeholders.
- The cross-cutting invariants in section 5 are binding throughout.

1. Roles & responsibilities

Role	Mandate	Owns	Gate it holds
Operator	Directs the project; the only one who pushes and gives GO; schedules session turns.	Product intent, roster, GO/NO-GO, pushes.	Every phase GO.
Coordinator	Runs the bus and the process.	CLAUDE.md §0 + coordination + command-registry, .coord/ bus, claim-map, commit serialization, push barrier, work-done map, the ready report.	P1–P5 gates.
Curator	External SME / source capture; certifies “core raised” by an object-store walk.	Curation channel, certification.	P1 core-raised certification.
Specialists	Module owners (backend, frontend, DB, devops, ...); do the work within their claims.	Their module paths.	Their READY/HOLD at barriers.
Security	Mandatory cross-cutting gate.	Security review.	Mandatory ACK before any prod release.
Tech Writer	Documentation gate and author.	docs/** authoring, DOCS inventory, approved/editing governance.	Doc-sync impact-triage ack in the push quorum.

2. The six-phase model

Each arrow crosses a verification gate confirmed by walking the repo / object store.

P0 Universe	->	P1 Core	->	P2 Roster & Claims	->	P3 Tree hygiene	->	P4 Work-
done map	->	P5 Ready						
(operator)		(coord)		(coord + operator)		(coord/CC)		
(coord)		(coord)						

3. Phases

P0 — Universe (the empty world)

Goal: a mounted, version-controlled project folder with a seed spec.

Who: Operator.

Actions:

- Create or connect one Cowork project folder; record its single absolute working path (the working-folder rule — all reads and writes use exactly that path).
- git init; create the working branch; pick a short session prefix (e.g. BY-AD) used by every session and chat.
- Seed CLAUDE.md with the product spec: product, stack, scope / out-of-scope, working-folder path, session prefix.
- Commit the baseline.

Artifacts: repo (.git), branch, seed CLAUDE.md, the working-folder rule.

Verification gate

git log shows the repo; the folder is mounted; CLAUDE.md is present (object-store check).

Operator GO: → build the core.

P1 — Core (discipline + bus)

Goal: the binding §0 discipline, the coordination protocol, and the .coord/ bus exist.

Who: Coordinator.

Actions:

- Extend CLAUDE.md §0 (BINDING) — see Appendix A: 0.1 verify; 0.2 resume-integrity; 0.3 Python + os.fsync writes, Edit tool BANNED on the mount; 0.4 pre-commit check; 0.5 mount .git/status UNRELIABLE → object-store verify; 0.6 NO auto-push and you cannot push from the mount [L-SC-20]; 0.7 post-commit re-sync from HEAD (PD-007).
- Add the multi-session coordination section + a command registry.
- Build the .coord/ bus (Appendix B).
- Write one init-prompt per planned specialist (Appendix C); tag not-yet-active roles FROZEN / NOT-YET-EXISTS.

Artifacts: CLAUDE.md §0 + coordination + command-registry; .coord/ bus; init-prompts.

Verification gate

The curator certifies by object store (not chat) — §0 anchors present, bus files exist, init-prompts carry the mandatory blocks.

Operator GO: → raise the roster.

P2 — Roster & Claims

Goal: a validated, non-overlapping claim-map.

Who: Coordinator (+ operator confirms the roster).

Actions:

- Operator confirms the initial roster.
- Build the claim-map (Appendix D): module → paths, seam rules, READ-ONLY / no-touch zones.
- Validate against the real tree: do the paths exist? nesting? overlaps?
- The roster is NOT frozen — the operator may split a role or activate one mid-bootstrap; re-validate on every change.
- Share a cross-cutting asset (e.g. DB migrations) by OWN / REVIEW / APPLY across three roles — never by carving a sub-path out of another owner's claim.

Artifacts: claim-map, seam rules.

Verification gate

real paths, ZERO overlap, a single owner per seam.

Operator GO: → tree hygiene.

P3 — Tree hygiene

Goal: a clean, intentional, committed tree.

Who: Coordinator issues CC tasks; specialists run them (native CC, not the mount).

Actions:

- Commit .gitignore FIRST, then re-check status.
- Decide tracked vs ignored BEFORE any blanket add: legacy / large dirs → .gitignore (or LFS by owner call); logs → ignore; DB dumps → owner call; word-lock ~\$ files → discard.
- De-duplicate documents ONLY by a verified subset proof (textual diff: drop X only if X is a subset of Y with zero unique lines).
- Commit existing work in labeled, module-grouped commits — native git / CC (the mount can't commit), serialized by commit.lock, journaled, with a post-commit re-sync from HEAD.

Artifacts: .gitignore, module-grouped commits, journal lines.

Verification gate

git status clean; every commit journaled.

Operator GO: → work-done map.

P4 — Work-done map

Goal: an honest map of what actually exists.

Who: Coordinator.

Actions:

- Build the map by walking the committed tree, labeling each item Intended / Drafted / Delivered (verified) / Committed (hash).
- Delivered (verified) requires a green build/test — a commit is not “Delivered”.
- Distribute the first tasks per the claim-map (via the dispatch loop, section 4).

Artifacts: work-done map, first task assignments.

Verification gate

the map is built from the repo, not chat; every “Delivered” is backed by a build/test.

Operator GO: → ready.

P5 — Ready

Goal: the team is up and the first work is moving.

Who: Coordinator.

Actions:

- Write the “team up & ready” report (Appendix E) — a Definition-of-Done checklist that includes the claim-map and confirms that each “Delivered” item was verified by the object store (git cat-file / git show / git hash-object), not chat.

Artifacts: the “team up & ready” report.

Verification gate

the report’s credibility is the object-store proof behind every “Delivered” claim, plus a defined push barrier + quorum (incl. the mandatory Security ack and the Tech-Writer doc-sync ack).

Operator GO: → the project is launched.

4. Operational loop — task dispatch (the heartbeat after launch)

Once the team is stood up (P5), this is how work actually moves. It is operator-mediated: the coordinator routes, the operator triggers, the specialist executes.

1. **The coordinator drafts** a full, self-contained directive (a compose-directive routing action — not a base session-coord §10 verb; a project may register its own): mandatory reads + the integrity block + the specialist’s claim + the task + acceptance criteria + commit.lock / journal / no-push. Code work points at a tasks/<file>.md (CC-only code rule).
2. **The coordinator writes** the directive into that specialist’s inbox .coord/inbox/<slug>.md (its router duty). It does not execute it and does not trigger the session.
3. **The coordinator hands the operator a trigger-list:** the session slugs to activate (e.g. backend-0611, security-0611).
4. **The operator runs the intake command** (коорд: входящие) in each named session. That session reads its own inbox inbox/<slug>.md and does the work via CC.
5. **The specialist reports back** to the coordinator via inbox/coordinator-<slug>.md; it marks a message handled only as its recipient.
6. **The coordinator** on its next intake reads its inbox, journals the outcome, and routes the next step.

Three roles in one line: Coordinator = router (stages directives, reviews, journals — writes no feature code, never “sends” by triggering). Operator = the trigger (activates each session). Specialist = executor (reads its own inbox, does the CC work, reports).

```
coordinator --writes directive--> inbox/<slug>.md      coordinator --trigger-
list--> OPERATOR
OPERATOR --intake in session--> specialist --reads own inbox, CC work--> commit
(commit.lock + journal, no push)
specialist --report--> inbox/coordinator-<slug>.md --> coordinator (next intake) --
journal--> next task
```

Verb precision. The registry lists the intake verb as the coordinator reading an inbox; operationally the operator uses the same intake verb in every session to mean “read my inbox and act” (in a specialist session, “my inbox” = inbox/<that-slug>.md). Cite as canonical only the verbs in the session-coord §10 registry; describe project-specific verbs as patterns or add them to §10.

Invariants of the loop

All code changes go out as CC prompts (CC-only rule); the coordinator analyses / plans / reviews / journals only. A handled-marker is written only by the recipient of a message. Commits run under commit.lock, are journaled, and never auto-push.

5. Cross-cutting invariants (binding throughout)

- **Verify, not chat** — tool success ≠ delivery; build status tables by walking the repo (§0.1).
- **Python + os.fsync for every write; the Edit tool is BANNED on the mount** (§0.3).
- **Object-store verification** — the mount’s .git / git status is unreliable; confirm with git hash-object vs git rev-parse HEAD:<file>; never escalate “corruption” from a mount read (§0.5).
- **No auto-push; you cannot push from the mount** — every push is a conscious operator decision via the dedicated push prompt [L-SC-20] (§0.6).
- **Post-commit re-sync from HEAD** — counteract the mount’s async cache write-back (PD-007, §0.7).
- **commit.lock covers the plumbing path too** — the commit-tree + direct ref-write path has no git-level locking; never bypass commit.lock.
- **Handled-markers are written only by the recipient** of a message.
- **The two-Cowork cross channel is deferred** until a second Cowork instance actually exists.
- **Documentation governance** — a coordinator-assigned Product Release ID scheme RTM-REL-YYYY.MM[.patch] + a Shipped-with column; a doc-sync impact-triage gate in the push quorum; and the approved/{doc,pdf} + editing/ layout with a mandatory revision-history table.

6. Case study & lessons — Agent Desktop bootstrap

Worked example. Project: Agent Desktop / Bynet — a Cisco UCCE/UCCX agent desktop via Finesse; .NET 8 backend, frontend added later. Captured by the curator; use it as the concrete illustration — the normative steps are sections 1–5 above.

Timeline

- **M0** — the curation channel is established, separate from the active coordinator bus; curator and project inboxes are set up.
- **M1** — curator Q&A; the claim-map is corrected against the real tree (3 fixes) plus a tree-hygiene precondition.
- **M2 (operator GO)** — core raised, verified by a curator object-store walk: CLAUDE.md v1.8 (1054 lines) with §0.5/0.6/0.7, the multi-session protocol, and the command registry; the .coord/ bus (14 files); 5 init-prompts (frontend tagged FROZEN + NOT-YET-EXISTS). Boundary honored — only .coord/ touched, no commit/push.
- **M3** — pre-hygiene curator review; init-prompts certified; tree-hygiene labeling guidance issued.
- **M4** — full roster up (7 sessions). The operator changes the roster mid-bootstrap: DBA/Devops split into two roles; frontend un-frozen (first task = a React + Vite + MUI scaffold); the prefix is locked BY-AD. A Step-0 gate passes: a duplicate spec doc is resolved by a textual-diff subset proof (a is a subset of c, zero unique) before deletion. The coordinator chat migrates mounts; live state is kept in coordinator_handoff.md.
- **M5 (in progress)** — tree hygiene is a CC job (the mount physically cannot commit: a stuck .git/index.lock, and bindfs denies rm/rename inside .git → [L-SC-20] proven). It awaits native CC for three module-grouped, labeled commits; then the work-done map (build + test before any “Delivered” label).

Lessons (generalize for any project)

7. The roster is not frozen at the roster phase — re-validate the claim-map for overlap on every operator change.
8. Share a cross-cutting asset by OWN / REVIEW / APPLY across roles, never by carving a sub-path out of an owner's claim.
9. De-duplicate documents only by a verified subset proof (X subset of Y, zero unique), never by assumption.
10. The mount cannot commit/push (stuck .git locks; bindfs denies rm/rename in .git) — all commits run native CC, serialized by commit.lock, journaled, with a post-commit re-sync. [L-SC-20].
11. Certify “core raised” by walking the repo via the object store, never from a chat claim.
12. A handled-marker is written only by the recipient of a message.
13. A cross-project channel that depends on both mounts breaks on remount — route each direction through a file in the writer's own bus, read by the party that holds both mounts (the curator).

Tree-hygiene numbers (from the worked example)

Dirty tree at start: 3 modified + 23 untracked. Sequence: commit .gitignore first → re-check; decide tracked-vs-ignored before any blanket add (legacy ~750 MB → .gitignore, no LFS; production logs → ignore; root SQL dumps → DBA call; word-lock ~\$ files → discard; spec docx → docs/); then module-grouped labeled commits; a build/test gate before labeling any feature “Delivered”.

Appendix A — CLAUDE.md skeleton

Copy and fill the placeholders. \$0 is binding.

```
# <PROJECT> — CLAUDE.md
> Single source of truth for autonomous work. Working folder: <ABSOLUTE WORKING
PATH>. Session prefix: <PREFIX>.

## 0. Environment rules — executor must read first (BINDING)
### 0.1 Verification — tool success != delivery
Verify via ls / git log / Read before claiming anything exists or is delivered.
Build status by walking the repo.
### 0.2 Session-resume integrity
On resume: git status --short; for each M file check truncation; restore via: git
show HEAD:<f> > <f>.
### 0.3 Writes: Python + os.fsync ONLY; the Edit tool is BANNED on the mount
Every write: read -> modify -> write with f.flush(); os.fsync(f.fileno()); then
verify with tail -3 and wc -l.
### 0.4 Pre-commit check
Run the pre-commit verifier before every git add / commit; on FAIL restore from HEAD
and retry.
### 0.5 Mount .git / git status is UNRELIABLE -> object-store verify
Confirm tracked-state and content via git hash-object <f> vs git rev-parse HEAD:<f>.
Never escalate corruption from a mount read.
### 0.6 No auto-push; you cannot push from the mount [L-SC-20]
Never git push automatically; pushes are an operator decision via the dedicated push
prompt.
### 0.7 Post-commit re-sync from HEAD (PD-007)
After every commit, re-sync committed files from HEAD as the last action
(counteracts cache write-back).

## <N>. Multi-session coordination
State bus in .coord/; claims by module; commit.lock serializes commits (covers the
plumbing path); journal every
commit; push barrier with quorum (mandatory Security ack + Tech-Writer doc-sync
ack). Handled-markers by recipient only.

## <N+1>. Command registry
<list each operator command verb and its exact semantics>
```

Appendix B — .coord/ bus files

```
.coord/
  README.md          # what the bus is + quick reference
  .gitignore         # ignore the runtime state below
  sessions/<slug>.md  # one per active session (frontmatter below)
  locks/commit.lock  # exclusive commit token (repo-wide)
  journal.md         # append-only commit log
  inbox/<slug>.md     # directed messages TO a session/role (coordinator-<slug>,
  curator, ...)
  coordinator_handoff.md # live resume artifact for a fresh coordinator
  push/
    request.md       # active barrier (freeze); tombstone when cleared
    acks/<slug>.md   # per-session READY / HOLD
```

Session file frontmatter:

```

---
session: <Name>
slug: <role>-<MMDD>
role: <role>
status: active          # active | pushing | done
modules: [<module>]      # claimed modules
files: []                # optional file-level claims
heartbeat: <UTC>
cc_task: none
---
<one-paragraph charter>

```

Inbox message block (handled-marker by the recipient only):

```

## <UTC> | from: <slug> | to: <slug>
<body>
---
> handled <UTC> by <recipient-slug> – <what was done>

```

Appendix C — Init-prompt template (per specialist)

```

<PREFIX>: you are the standing <ROLE> specialist. Session name "<Name>".
0. COMMS: your inbox = .coord/inbox/<slug>.md (read it whole; mark each block
handled). Your outbox = the
  COORDINATOR's SLUG inbox .coord/inbox/coordinator-<slug>.md. All .coord/ and work
writes via Python + os.fsync.
  Commands per the session-coord skill section 10 – never guess an unknown verb.
1. INTEGRITY (0.2): git status; for M files hash-object vs HEAD; restore truncated.
2. MATERIAL: read CLAUDE.md section 0 + coordination + your domain sections.
3. TERRITORY: you OWN <paths> (claims); READ-ONLY elsewhere; shared seams via OWN /
REVIEW / APPLY.
4. REGISTER: write .coord/sessions/<slug>.md, status active.
5. READ BUS: your inbox; sessions/*; journal tail; push/request (if FREEZE -> stop).
6. PUSH QUORUM: give READY / HOLD on your paths at every barrier.
7. CONFIRM: flush an ack to the coordinator's slug inbox + a short chat reply.

```

Appendix D — Claim-map template

```

# Claim-map – <PROJECT>
| Module | Owner (slug) | Paths (claims) | Seam rule | Read-only / no-touch |
|---|---|---|---|---|
| <module> | <role>-<MMDD> | <paths> | OWN / REVIEW / APPLY where shared | <paths
others own> |

```

Validation: every path exists in the real tree; ZERO overlap; one owner per seam.
 Re-validate on any roster change.

Appendix E — “Team up & ready” report template

```
# Team up & ready – <PROJECT> (<date>)
Definition of Done:
- [ ] Core: CLAUDE.md section 0 (0.1-0.7) + coordination + command-registry –
object-store verified.
- [ ] Bus: .coord/ files present (README, sessions/, locks/, journal, inbox/,
push/).
- [ ] Roster holding: sessions registered; claim-map attached; ZERO overlap.
- [ ] Tree clean: git status empty; commits journaled.
- [ ] Work-done map attached; EACH “Delivered” verified by object store (cat-file /
show / hash-object), not chat.
- [ ] First tasks issued per the claim-map.
- [ ] Push barrier + quorum defined, incl. MANDATORY Security ack (+ Tech-Writer
doc-sync ack).
- [ ] First CC task ran clean.
Operator G0: _____
```

Sources: *in-repo CLAUDE.md §0 / §42 / §44, the live .coord/ bus, and the session-coord skill (normative methodology); the Agent Desktop bootstrap capture by the curator (case study). EN canonical; a ru edition follows on request.*